Published: 01 December 1979

ARTICLE

High-Order Fast Elliptic Equation Solvers

E. N. HOUSTIS, Purdue University, West Lafayette, IN, United States

T. S. PAPATHEODOROU, Clarkson University, Potsdam, NY, United States

High-Order Fast Elliptic Equation Solver

E. N. HOUSTIS
Purdue University
and
T. S. PAPATHEODOROU
Clarkson College of Technology

An algorithm for approximating certain classes of elliptic partial differential equations on a rectangle is presented. The algorithm uses high-order 9-point difference approximations to the Helmholtz-type (fourth-order) or Poisson (sixth-order) equations and the fast Fourier transform. Compared to efficient second-order fast direct methods for smooth problems, the execution time is reduced by a large factor, typically 50 for the Helmholtz-type equations and over 100 for the Poisson problem. Comparisons with two high-order fast direct methods indicate the superiority of the algorithm.

Key Words and Phrases: fast Fourier transform, high-order methods, fast Helmholtz solver, fast Poisson solver
CR Categories: 5.17
The Algorithm: FFT9, Fast Solution of Helmholtz-Type Partial Differential Equations. *ACM Trans. Math. Software 5*, 4 (Dec. 1979), 490–493.

1. INTRODUCTION

An algorithm for approximating two particular classes of elliptic partial differential equations is implemented in a Fortran program FFT9 [8]. This piece of software computes a discrete fourth-order approximation to the solution u of the Helmholtz-type equation

$$\alpha u_{xx} + \beta u_{yy} + \lambda u = f, \qquad \Omega = [a, b] \times [c, d] \tag{1.1}$$

subject to Dirichlet boundary conditions

$$u = g \text{ on } \partial\Omega \equiv \text{boundary of } \Omega \tag{1.2}$$

where α, β, and λ are real constants.

For Poisson equations a sixth-order discretization procedure is implemented. The method uses a 9-point difference approximation to the Helmholtz-type equation, (1.1), with constant coefficients developed by Houstis and Papatheodorou [7] or a sixth-order discretization of the Poisson equation derived by Lynch [10]. The resulting difference equations are solved by a combination of a block

This work was supported in part by the National Science Foundation under Grant MCS 76-10225.
Authors' addresses: E. N Houstis, Department of Computer Science, Purdue University, West Lafayette, IN 47907; T. S. Papatheodorou, Department of Mathematics, Clarkson College of Technology, Potsdam, NY 13676

cyclic reduction and Fourier analysis. Hockney [3] has devised and implemented a similar direct method (the FACR algorithm) with 5-point second-order difference approximation to Poisson equations. FFT9 uses Hockney's implementation [4] of the algorithm for the evaluation of the fast Fourier transform. A survey of other fast methods that could be adapted for the solution of the difference equations is given by Dorr [2].

In Section 2 we describe the algorithm and its modular structure. In Section 3 we present a comparison of FFT9 with two high-order fast direct methods—a fourth-order Galerkin's tensor product and a fourth-order block cyclic reduction [5]. A comparison of this algorithm with other well-known fast direct methods is given in [7] for the Helmholtz-type equation. For completeness, we selectively report in this paper a few data, presented in [7], comparing FFT9 with a second-order method (NCAR). All test results show the superiority of FFT9.

2. METHOD

2.1 Grid Module

A uniform rectangular grid is placed over the rectangular region Ω. The number of vertical and horizontal mesh lines is $N_x = 2^k + 1$ and $N_y = 2^l + 1$, respectively, where k and l are integers.

2.2 Discretization Modules

The new fourth-order 9-point finite difference approximation to eq. (1.1) used is

$$p = \frac{\Delta y}{\Delta x}, \qquad \gamma = \frac{(\Delta x)^2}{12}, \qquad q = \frac{\beta}{\alpha}\frac{1 - \lambda(\Delta y)^2/12\beta}{1 - \gamma\lambda/\alpha},$$

$$a = 12\frac{\beta}{\alpha}\frac{1}{p^2 + q} = -2, \qquad b = \frac{12p^2q\alpha/\beta}{p^2 + q} - 2,$$

$$c = (b + 2)(1 - \lambda\gamma/\alpha)\lambda(\Delta x)^2/\alpha - 2(a + b) - 4,$$

$$d = \alpha q/\beta \quad \text{and} \quad e = 12(1 - \lambda\gamma/\alpha)d - 2d - 2$$

$$\begin{bmatrix} 1 & b & 1 \\ a & c & a \\ 1 & b & 1 \end{bmatrix} u = \frac{p^2\Delta x^2/\alpha}{p^2 + q} \begin{bmatrix} 0 & 1 & 0 \\ d & e & d \\ 0 & 1 & 0 \end{bmatrix} f. \tag{2.1}$$

The boxes contain the weights to be applied on the 9-point stencil. The difference equation of the new sixth-order discretization to solutions of the Poisson equation is

$$1/(6h^2)\begin{bmatrix} 1 & 4 & 1 \\ 4 & -20 & 4 \\ 1 & 4 & 1 \end{bmatrix} u = (1/360)\begin{bmatrix} 1 & 0 & 4 & 0 & 1 \\ 0 & 48 & 0 & 48 & 0 \\ 4 & 0 & 148 & 0 & 4 \\ 0 & 48 & 0 & 48 & 0 \\ 1 & 0 & 4 & 0 & 1 \end{bmatrix} f \tag{2.1}'$$

where the stencil for the right-hand side is expanded at the half-lattice points with the main lattice having separating $h = \Delta x = \Delta y$.

2.3 Equation Solution Module

The system of difference equations formed in the discretization module after the elimination of boundary conditions can be written as

$$\begin{bmatrix} \Gamma & \Delta & & & & \\ \Delta & \Gamma & & & & \\ & \ddots & \ddots & & & \\ & & \ddots & \ddots & & \\ & & & \Gamma & \Delta \\ & & & \Delta & \Gamma \end{bmatrix} \begin{bmatrix} \mathbf{u}_1 \\ \mathbf{u}_2 \\ \vdots \\ \mathbf{u}_{N-1} \\ \mathbf{u}_N \end{bmatrix} \quad \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \vdots \\ \mathbf{b}_{N-1} \\ \mathbf{b}_N \end{bmatrix}$$

where the matrices Δ and Γ are

$$\Delta = \text{tridiag } (1, a, 1)$$
$$\Gamma = \text{tridiag } (b, c, b)$$

and $N = N_x - 1 = N_y - 1$ was assumed for simplicity. Note that matrices Γ, Δ are commutative, i.e. $\Gamma\Delta = \Delta\Gamma$. The vectors

$$\mathbf{u}_j = (u_{1j}, \ldots, u_{Nj})^T, \qquad \mathbf{b}_j = (b_{1j}, \ldots, b_{Nj})^T$$

are the solution and right-hand side for the jth column of the mesh. The formation of the right-hand side $\mathbf{b}_j$ is performed by the subroutine RGTSD (see [8]).

We now describe the steps of the algorithm used to solve the difference equations.

Step 1: *Odd/Even Reduction.* Consider three consecutive blocks of difference equations:

$$\begin{aligned} \Delta\mathbf{u}_{j-2} + \Gamma\mathbf{u}_{j-1} + \Delta\mathbf{u}_j &= \mathbf{b}_{j-1} \\ \Delta\mathbf{u}_{j-1} + \Gamma\mathbf{u}_j + \Delta\mathbf{u}_{j+1} &= \mathbf{b}_j \\ \Delta\mathbf{u}_j + \Gamma\mathbf{u}_{j+1} + \Delta\mathbf{u}_{j+2} &= \mathbf{b}_{j+1} \end{aligned}$$

where j is even.

Multiply line $j - 1$ by Δ, line j by Γ, line $j + 1$ by Δ, and add them to obtain

$$\Delta^2\mathbf{u}_{j-2} + (2\Delta^2 - \Gamma^2)\mathbf{u}_j + \Delta^2\mathbf{u}_{j+2} = \mathbf{b}_j^*$$

where the right-hand-side vector is $\mathbf{b}_j^* \equiv \Delta(\mathbf{b}_{j-1} + \mathbf{b}_{j+1}) - \Gamma\mathbf{b}_j$. The formation of the vectors $\mathbf{b}_j^*$ is performed by subroutine EVENRD (see [8]).

Step 2: *Fourier Analysis.* A real finite Fourier transformation is performed on the vectors $\mathbf{u}_j$ and $\mathbf{b}_j^*$

$$\mathbf{b}_j^* = \sum_{k=1}^{N} b_{jk}^* \mathbf{V}_k, \qquad \mathbf{u}_j = \sum_{k=1}^{N} \phi_{jk} \mathbf{V}_k \tag{2.2}$$

where $\mathbf{V}_k = \sqrt{(2/(N+1))}(\sin k\theta, \sin 2k\theta, \ldots, \sin(Nk\theta))^T$, $k = 1, \ldots, N$, and $\theta = \pi/(N+1)$. The vectors $\mathbf{V}_k$, $k = 1, \ldots, N$, are linearly independent and are

Table I. Data for Solving $u_{xx} + u_{yy} = f$, $u = 0$ on Unit Square with u Taken as $3e^x e^y(x - x^2)(y - y^2)$ (The estimated order of convergence is given as order.)

N =	4	8	16	32	64	128
Method:	FFT9 (fourth order)					
Maximum error		6.82E − 05	4 27E − 06	2.68E − 07	1.68E − 08	1.04E − 09
Time (seconds)		0.05	0 19	0.76	3.13	13.03
Order		4.0	4 0	4.0	4.0	4.0
Method:	FFT9 (sixth order)					
Maximum error		1.34E − 07	2.10E − 09	3.3E − 11	9.49E − 13	8.42E − 13
Time (seconds)		0.07	0.28	1.14	4.66	19.16
Order		6.0	6.0	5.0	(roundoff effects)	
Method·	GLFFTP (fourth order)					
Maximum error	2.92E − 04	2.28E − 05	1.48E − 06	9.23E − 08		
Time (seconds)	0 79	3.18	12.87	52.41		
Order		4.0	4.0	4.0		
Method:	BUNPT (fourth order)					
Maximum error		6.82E − 05	4.27E − 06	2.68E − 07	1.68E − 08	1.05E − 09
Time (seconds)		0.05	0.21	0.90	3.93	16.95
Order		4.0	4.0	4.0	4.0	4.0

the eigenvectors of Γ, Δ. The corresponding eigenvalues are

$$\lambda_k(\Delta) \equiv a + 2\cos(k\theta), \qquad \lambda_k(\Gamma) = c + 2b\cos(k\theta), \qquad k = 1, \ldots, N.$$

The Fourier coefficients

$$b^*_{jk} = \mathbf{b}_j^{*\mathrm{T}} \cdot \mathbf{V}_k = \sum_{i=1}^{N} b^*_{ji} V_{ki}$$

for each even j are computed by subroutine FOUR which performs the fast summation (see [4]). The Fourier coefficients ϕ_{jk} satisfy the equations

$$\phi_{j-2,k} + (2 - (\lambda_k(\Gamma)/\lambda_k(\Delta))^2)\phi_{j,k} + \phi_{j+2,k} = b^*_{jk}/(\lambda_k(\Delta))^2 \tag{2.3}$$

for $k = 1, \ldots, N$. The N systems, eq. (2.3), are solved by the subroutine CRED (see [8]) which performs recursive cyclic reduction (see [4]).

Step 3: *Fourier Synthesis.* After the determination of the Fourier coefficients by solving eq. (2.3) the solution vectors $\mathbf{u}_j$ on even lines is computed using subroutine FOUR (see [8]) and the transformation equation, eq. (2.2).

Table II. Data for Solving $u_{xx} + u_{yy} = f$, $u = 0$ on Unit Square with u Taken as $x^{5/2}y^{5/2} - xy^{5/2} - x^{5/2}y + xy$
(The third derivatives of the solution have singularities at x or $y = 0$.)

N =	4	8	16	32	64	128
Method:	FFT9 (sixth order)					
Maximum error		1.58E − 05	3.18E − 06	6.06E − 07	1.11E − 07	2.00E − 08
Time (seconds)		0.06	0.24	0.97	3.99	16.42
Order			2.3	2.4	2.45	2.47
Method	FFT9 (fourth order)					
Maximum error		1.29E − 04	2.53E − 05	4.80E − 06	8.80E − 07	1.59E − 07
Time (seconds)		0.04	0.17	0.67	2.77	11.59
Order			2.4	2.4	2.4	2.5
Method:	GLFFTP (fourth order)					
Maximum error	2.21E − 05	2.65E − 06	4.30E − 07	7.29E − 08		
Time (seconds)	0.60	2.43	9.81	39.71		
Order		3.06	2.62	2.56		
Method:	BUNPT (fourth order)					
Maximum error		1.30E − 04	2.54E − 05	4.80E − 06	8.80E − 07	1.59E − 07
Time (seconds)		0.05	0.18	0.80	3.56	15.57
Order			2.4	2.4	2.4	2.5

Step 4: *Solution on Odd Lines.* Consider the difference equations for j = odd, i.e.

$$\Delta \mathbf{u}_{j-1} + \Gamma \mathbf{u}_j + \Delta \mathbf{u}_{j+1} = \mathbf{b}_j$$

or

$$\Gamma \mathbf{u}_j = \mathbf{b}_j - \Delta(\mathbf{u}_{j+1} + \mathbf{u}_{j-1}). \tag{2.4}$$

The right-hand side of the above triangular systems is computed by subroutine ODDRD (see [8]) and the systems are solved by the subroutine CRED which determines the solution of eq. (2.4) on odd lines.

3. TEST RESULTS

This algorithm with the fourth-order discretization procedure has been compared in [7] with a second-order 5-point block cyclic reduction method from the NCAR

Table III. Data for Solving $4u_{xx} + u_{yy} - 64u = f$, $u = 0$ on Unit Square with u Taken as $4(x^2 - x)(\cos(2\pi y) - 1)$
(The solution is smooth.)

N =	4	8	16	32	64	128
Method:	FFT9 (fourth order)					
Maximum error		1.43E − 03	9.27E − 05	5.85E − 06	3.66E − 07	2.29E − 08
Time (seconds)		0.04	0 17	0.69	2.82	11.53
Order			3.9	4.0	4 0	4.0
Method:	GLFFTP (fourth order)					
Maximum error	3.38E − 03	4 07E − 04	3 09E − 05	2.03E − 06		
Time (seconds)	0.66	2.72	11.01	44 62		
Order		3.05	3.72	3.93		

Table IV. Data for Solving $u_{xx} + u_{yy} - 100u = 0$, $u = g$ on the Unit Square with u Taken as $(\cosh 10x + \cosh 10y)/\cosh 10$
(The solution has boundary layer type behavior.)

N =	4	8	16	32	64	128
Method:	FFT9 (fourth order)					
Maximum error		1 61E − 03	1 17E − 04	7.63E − 06	4.82E − 07	3.02E − 08
Time (seconds)		0.04	0.14	0.52	2 06	8 54
Order			3.8	3.9	4.0	4.0
Method.	BUNPT (fourth order)					
Maximum error		2.89E − 03	1.83E − 04	1.16E − 05	7.25E − 07	4.54E − 08
Time (seconds)		0.06	0.18	0.71	2.99	12 95
Order			4.0	4.0	4.0	4.0

package [13], 5- and 9-point tensor product methods (see [11, 12]) with or without fast summation. The algorithms were compared for a set of eight elliptic boundary value problems used by Houstis et al. in [6] for the evaluation of methods for the solution of elliptic partial differential equations. These comparisons show that FFT9 is on the average 50 times faster.

In this section we compare FFT9 with the sixth-order discretization procedure for Poisson problems, Galerkin with Hermite bicubics-tensor product method

Table V. Data for Solving $u_{xx} + u_{yy} - 100u = f$, $u = g$ on the Unit Square with u Taken as $\cosh 10x/\cosh 10 + \cosh 20y/\cosh 20$ Having Sharper Boundary Layer Behavior Than the Previous Example

N =	4	8	16	32	64	128
Method:	FFT9 (fourth order)					
Maximum error		1.94E − 02	1.81E − 03	1.20E − 04	7.63E − 06	4.81E − 07
Time (seconds)		0 07	0.22	0.84	3.25	13.0
Order			3.4	3.9	4.0	4.0
Method	BUNPT (fourth order)					
Maximum error		3.67E − 02	3.19E − 03	2.08E − 04	1.31E − 05	8.27E − 07
Time (seconds)		0.1	0.34	1.27	5.09	21.20
Order			3.5	3.9	4.0	4.0

Table VI. Data for Solving $u_{xx} + u_{yy} = f$, $u = g$ on the Unit Square with u Taken as $\phi(x) * \phi(y)$ Where $\phi(x) = u(0) - u(0.35) + (u(0\,35) - u(0.65))p(x)$, $p(x)$ Is a Quintic Polynomial Determined So That $\phi(x)$ Has Two Continuous Derivatives and $u(x)$ Is a Unit Step Function (The solution has a double wave front.)

N =	4	8	16	32	64	128
Method.	FFT9 (fourth order)					
Maximum error		1.12E − 01	5.17E − 02	4.09E − 03	2.96E − 03	5.70E − 05
Time (seconds)		0.08	0.27	1.01	4.07	16.55
Order			1.1	3.7	0.5	5.7
Method:	FFT9 (sixth order)					
Maximum error		4.56E − 02	7.28E − 03	3.43E − 03	2.53E − 04	2.18E − 04
Time (seconds)		0 11	0.41	1.65	6.66	26.33
Order			2.65	1.09	3.76	0 21
Method·	BUNPT (fourth order)					
Maximum error		1.12E − 01	5.17E − 02	4.09E − 03	2.96E − 03	5.69E − 05
Time (seconds)		0.08	0.29	1.16	4.91	20 93
Order			1.1	3.7	0.5	5.7

Table VII. Data for Solving $u_{xx} + u_{yy} = f$, $u = g$ on the Unit Square with u Taken as $y[(x-2)^2 + y^2 - 1]e^{-0.0625(x-4)(y-2)}/[(3+(x-2)^2)(3+y^2)]$

N =	4	8	16	32	64	128
Method:	FFT9 (fourth order)					
Maximum error		5.03E − 03	2.88E − 04	1.77E − 05	1.10E − 06	6.89E − 08
Time (seconds)		0.15	0.53	2.05	8.05	32.12
Order			4.1	4.0	4.0	4.0
Method:	FFT9 (sixth order)					
Maximum error		2.74E − 05	3.40E − 07	5.11E − 09	7.82E − 11	6.37E − 12
Time (seconds)		0.24	0.89	3.54	14.13	56.94
Order			6.33	6.06	6.03	3.62
Method:	BUNPT (fourth order)					
Maximum error		5.03E − 03	2.88E − 04	1.77E − 05	1.10E − 06	6.90E − 08
Time (seconds)		0.14	0.54	2.16	8.78	36.16
Order			4.1	4.0	4.0	4.0

(GLFFTP) for eq. (1.1) with homogeneous boundary conditions, and a fourth-order cyclic reduction method for Helmholtz equations (BUNPT) [5]. For a description of the Galerkin-tensor product method for Poisson equations, see [1]. We extended and implemented this procedure for Helmholtz-type problems. Tables I through IX show the experimental results. The order of convergence is estimated by

$$\alpha_{m,n} = \ln(\epsilon_n/\epsilon_m)/\ln(n/m)$$

where ϵ_n is the estimated error for an $n \times n$ mesh. All computations were performed on a CDC 6500 in single precision arithmetic.

In Tables X and XI we present data comparing a second-order method (NCAR) against FFT9 and a fourth-order Galerkin-tensor product (GLFFTP), while in Table XII a comparison is made between GLFFTP and FFT9.

The data indicate, for smooth Poisson problems, that $N = 8$ with FFT9 gives better accuracy than $N = 128$ with the second-order (NCAR) and FFT9 provides an increase in efficiency by a factor 100 to 300, depending on the complexity of the function f.

It is worth noting that for Problem 2 with the solution having singularities in the third derivative, FFT9 (sixth order) is 78 times faster than NCAR (second order). While for Problem 6 whose solution has a double first and discontinuous

Table VIII. Data for Solving $u_{xx} + u_{yy} = f$, $u = 0$ on the Unit Square with u Taken as $10*\phi(x)*\phi(y)$, $\phi(x) = e^{-100(x-0.5)^2}(x^2 - x)$
(The solution has a sharp peak.)

N =	4	8	16	32	64	128
Method.	FFT9 (fourth order)					
Maximum error		3.55E − 01	7.53E − 03	3.81E − 04	2.27E − 05	1.40E − 06
Time (seconds)		0 14	0.52	2.03	8.07	32.46
Order			5.6	4.3	4.1	4 0
Method:	FFT9 (sixth order)					
Maximum error		1.77E − 02	1.07E − 04	1.26E − 06	1 92E − 08	2.94E − 10
Time (seconds)		0.24	0.92	3.59	14.32	57.49
Order			7.37	6.41	6.04	6.03
Method·	GLFFTP (fourth order)					
Maximum error	8.06E − 02	1.21E − 02	1.16E − 03			
Time (seconds)	3.44	13.81	55.37			
Order		2.74	3.38			
Method	BUNPT (fourth order)					
Maximum error		3.55E − 01	7.53E − 03	3.81E − 04	2.27E − 05	1.40E − 06
Time (seconds)		0 14	0.53	2.14	8.78	36.16
Order			5.6	4.3	4.1	4.0

third derivative, FFT9 (sixth order) resolves the execution time by a factor 3.4 compared to NCAR. Data in Table XIII indicate that FFT9 (sixth order) is on the average 17 times faster than BUNPT (fourth order) except for Problem 6.

The data in Table XI indicate that GLFFTP is faster than the 5-point NCAR program except for problems with the complicated right-hand sides. We observed that GLFFTP spends 80–97 percent of its execution time in calculating the right-hand side of the Galerkin equations. Data in Table XII indicate that FFT9 is on the average 28 times faster than GLFFTP. Finally, the test results in Tables I to IX show that FFT9 is faster and more accurate (for large N) than the block cyclic reduction Helmholtz equation solver BUNPT.

A systematic performance evaluation of high-order methods in the ELLPACK system (including FFT9) over "singular" problems is presented in [9].

Table IX. Data for Solving $u_{xx} + u_{yy} = f$, $u = g$ on the Unit Square with u Taken as $e^{x+y}/(1 + xy)$ (The solution has singularity near domain.)

N =	4	8	16	32	64	128
Method:	FFT9 (fourth order)					
Maximum error		9.30E − 06	5 85E − 07	3.66E − 08	2.29E − 09	1.47E − 10
Time (seconds)		0.06	0.22	0.86	3.49	14.19
Order			3.99	4.0	4.0	3.96
Method·	FFT9 (sixth order)					
Maximum error		4.76E − 08	7.94E − 10	1.49E − 11	2.81E − 12	5.95E − 12
Time (seconds)		0.09	0.33	1.31	5.23	21.19
Order			5.91	5.74	2.41	
Method:	BUNPT (fourth order)					
Maximum error		9.30E − 06	5.85E − 07	3.66E − 08	2.30E − 09	1.66E − 10
Time (seconds)		0.06	0.24	0.98	4 17	17.92
Order			3.99	4.0	4.0	3 79

Table X. Data Comparing 5-Point Cyclic Reduction (NCAR) with the FFT9 Sixth-Order Poisson Problems for the Same Accuracy

	NCAR (second order)			FFT9 (sixth order)		
Problem	N	Accuracy	Execution time	N	Accuracy	Execution time
1	128	1.70E − 05	18.69	8	1.34E − 07	0.07
2	128	1.28E − 06	18.72	16	3.18E − 06	0.24
6	128	5.05E − 04	22.41	64	2.53E − 04	6 66
7	128	2.02E − 04	36.95	8	2.74E − 05	0.24
8	128	9.23E − 04	37.80	16	1.07E − 04	0.92

Table XI. Data Comparing 5-Point Reduction (NCAR) with the Galerkin-Tensor Product (GLFFTP) for the Same Accuracy

	NCAR (second order)			GLFFTP (fourth order)		
Problem	N	Accuracy	Execution time	N	Accuracy	Execution time
1	128	1.70E − 05	18 69	8	2.28E − 05	3.18
2	128	1.28E − 06	18.72	8	2.26E − 06	2 43
3	128	5.73E − 05	17 70	16	3.09E − 05	11.01
8	128	9.23E − 04	37 80	16	1.16E − 03	55.37

Table XII Data Comparing Galerkin-Tensor Product (GLFFTP) with the FFT9 Fourth Order for the Same Accuracy

	GLFFTP (fourth order)			FFT9 (fourth order)		
Problem	N	Accuracy	Execution time	N	Accuracy	Execution time
1	32	9.23E − 08	52.41	64	1.68E − 08	3.13
2	32	7.29E − 08	39.71	128	1.59E − 07	11.59
3	32	2.03E − 06	44.62	32	5.85E − 06	0.69
8	16	1.16E − 03	55.37	32	3 81E − 04	2.03

Table XIII. Data Comparing Fourth-Order BUNPT with the FFT9 Sixth Order for the Same Accuracy

	BUNPT (fourth order)			FFT9 (sixth order)		
Problem	N	Accuracy	Execution time	N	Accuracy	Execution time
1	128	105E − 09	16.95	16	2.10E − 09	0.28
2	128	1.59E − 07	15.57	64	1.11E − 07	3.99
6	128	5 69E − 05	20.93	64	2 53E − 04	6.66
7	128	6.90E − 08	36.16	32	5.11E − 09	3.54
8	128	1.40E − 06	36.16	32	1.26E − 06	3.59

REFERENCES

1. Bank, R.E. Efficient algorithms for solving tensor product finite element equations. (Submitted to a technical journal.)
2. Dorr, F.W. The direct solution of the discrete Poisson equation on a rectangle. *SIAM Rev. 12* (1970), 248–263.
3. Hockney, R.W A fast direct solution of Poisson's equation using Fourier analysis. *J. ACM 12*, 1 (Jan 1965), 95–113.
4. Hockney, R.W. The potential calculation and some applications. *Meth. in Comput. Phys. 9* (1970), 135–211
5. Holden, E Solution of the nine-point operator by cyclic reduction. SUIPR Rep. 619, Stanford U , Stanford, Calif., March 1975.
6. Houstis, E N., Lynch, R.E., Papatheodorou, T.S., and Rice, J.R. Evaluation of numerical methods for elliptic partial differential equations *J. Comput. Phys 27* (1978), 323–349.
7. Houstis, E N., and Papatheodorou, T S. Comparison of fast direct methods for elliptic problems Proc. IMACS (AICA) Int. Symp. on Computer Methods for Partial Differential Equations, Lehigh U., Bethlehem, Pa., June 1977.
8. Houstis, E.N., and Papatheodorou, T.S. Algorithm 543, FFT9: Fast solutions of Helmholtz-type partial differential equations. *ACM Trans. Math. Software 5*, 3 (Sept. 1979), 490–493.
9. Houstis, E.N , and Rice, J.R. Performance evaluation of high order methods over 'singular' problems (Submitted to a technical journal.)
10. Lynch, R.E. $0(h^6)$ accurate difference approximation to solutions of the Poisson equation in three variables. CSD-TR 221, Purdue U , West Lafayette, Ind., 1977.
11. Lynch, R.E., Rice, J.R , and Thomas, D.H. Tensor product analysis of partial difference equations. *Bull Amer. Math. Soc. 70* (1964), 378–384.
12. Lynch, R E , Rice, J.R., and Thomas, D.H. Direct solution of partial difference equations by tensor product methods. *Numer Math. 6* (1964), 185–199.
13. Swarztrauber, P., and Sweet, R. Efficient FORTRAN subprograms for the solution of elliptic partial differential equations. NCAR-TN/IA-109, 1975.

Received May 1977, revised January 1979